juliacon

CliffordNumbers.jl: a Julia package for numeric primitives for geometry

Brandon S. Flores[1, 2] and Victor M. Zavala[2]

[1]Department of Chemistry, University of Wisconsin-Madison
[2]Department of Chemical and Biological Engineering, University of Wisconsin-Madison

ABSTRACT

Vector algebra, with its familiar cross product and dot product operations, has long been the *lingua franca* of physics and 3D geometry, mostly replacing the older quaternion formalism. While vector algebra managed to clean up some of the informal aspects of quaternions, it also introduced its own issues, particularly with the cross product, and remains limited to describing 3D space like the quaternions. An alternative formalism, *geometric algebra*, or *Clifford algebra*, generalizes the geometric concepts of vector algebra, quaternions, complex numbers, and dual numbers to any number of dimensions, and offers greater conceptual clarity for learners. In service of the goal of making geometric algebra more accessible to the broader community, `CliffordNumbers.jl` provides a first-class numerical implementation of geometric algebras in Julia. Here, we provide an introduction to geometric algebra for those not familiar with the topic, and provide details about the architecture of `CliffordNumbers.jl`, as well as usage examples.

Keywords

Julia, Geometry, Geometric algebra, Clifford algebra, Exterior algebra, Numerical libraries

1. Introduction

One of the great challenges of modeling physical systems is the treatment of high dimensions. Historically, a couple of major conventions have been used to model 3D space. In 1843, William Rowan Hamilton famously inscribed the formula $i^2 = j^2 = k^2 = ijk = -1$ into the stone of Brougham Bridge with a pocket knife, defining the quaternions [21]. Quaternions can represent/encode both scalar quantities and 3D vectors in a single object, and they permit addition, multiplication, and their inverses. Although the quaternions immediately found use in the physical sciences, notably in Maxwell's first publication of his famous equations, they were also considered unwieldy: Maxwell originally published 20 equations instead of the four we know today. Part of this unwieldiness stems from the noncommutativity of quaternion multiplication, but this property is necessary to describe rotations in 3D. More troubling was that 3D vectors would always have negative squares, which requires careful manipulation of signs to ensure that scalar quantities are correctly computed [4].

The deficiencies of quaternions led Josiah Willard Gibbs to develop a new formalism, vector algebra, which is the dominant formalism in all but a few niche applications today [18]. Rather than a single quaternion product, vector algebra uses two products, the dot and cross product, splitting the commutative and anticommutative components of the quaternion product into two different operations. However, vector algebra is not free of unwieldiness, either, particularly with the cross product. The cross product of two vectors does not transform like the input vectors under a reflection or other transformation that reverses the orientation of the space. For clarity, the output of the cross product is often called a *pseudovector*, and the dot product of a pseudovector with a vector (as in the triple product) is called a *pseudoscalar*. The vector algebra formalism requires these distinctions to correctly describe how multidimensional quantities transform.

Today, most scientific literature is written in the language of vector calculus, a combination of calculus and vector algebra. By contrast, quaternions see relatively little practical use, with the most notable exception being the description of 3D rotations. However, both quaternions and vector algebra also share one glaring deficiency: they are limited to describing 3D space. This is obvious from the limited size of quaternions, but even vector algebra is deficient in this regard without combining it with matrix algebra. It is possible to generalize the cross product to an $(n-1)$-ary operator on n-dimensional vectors, but this implies that the binary nature of the cross product is dependent on three dimensions. A seven-dimensional generalization exists, but does not satisfy the Jacobi identity [26].

While Hamilton's quaternions and Gibbs' vector algebra battled for dominance in the scientific community, a third formalism developed in the shadows. In 1844, Hermann Grassmann introduced *Ausdehnungslehre* (*Theory of Extension*), which we call *exterior algebra* today [19]. This algebra combines k, d-dimensional vectors into new objects called *k-vectors* (with the *grade*, k, ranging from 0 to d), with $\binom{d}{k}$ components, using a new operation, the *wedge product* ($\wedge$). With the wedge product, objects of any dimension from 0 to d residing in a d-dimensional space can be rigorously described. In 3D, the wedge product of two vectors is calculated exactly as the cross product is, but the result is a not an ordinary vector, but a 2-vector (or bivector). This distinction explains why axial vectors and pseudovectors transform differently in the vector calculus formalism: axial vectors are 1-vectors, and pseudovectors are 2-vectors (or more generally $(d-1)$-vectors in d dimensions).

In 1878, William Kingdon Clifford, familiar with the work of Hamilton and Grassmann realized that the "vector" components of quaternions actually transformed like the bivectors of a three-dimensional exterior algebra. He then constructed a new family of algebras, which he called *n-way geometric algebras*, which ex-

tended the exterior algebra to include quaternion-like constructions which worked in any number of dimensions [6]. Today, these algebras are called *Clifford algebras* in his honor.
Clifford's geometric algebra, as he formulated it, faded into obscurity. Notably, Paul Dirac used the geometric algebra of spacetime in his derivation of his eponymous equation, but phrased it in the language of matrices. Most literature on Clifford algebras uses the matrix-based formalism of representation theory, unlike Clifford's approach which treated Clifford algebras as a hypercomplex number system. David Hestenes advocated for a return to Clifford's original geometric approach, and used this framework to develop new approaches to geometric and physical problems [22] [23]. Although geometric algebra is still a niche formalism today, there has been a recent resurgence of interest in the topic: in part due to potential increases in computational efficiently, but primarily due to the conceptual and pedagogical clarity of the approach and because of increasing interest in computational geometry and topology.

Statement of need

The first computational tool we turn to when solving multidimensional problems is usually numerical linear algebra. Numerous libraries exist to efficiently implement fundamental matrix and vector operations and with high numerical stability. However, linear algebra may not be the best mathematical tool for phrasing a solution to a problem in low-dimensional geometry, and in some cases it may be an inferior approach. A prime example is implementing 3D rotations and reflections: while they can be composed and performed with matrix-vector multiplication, it has suboptimal efficiency, is not numerically stable, and can introduce shearing that does not respect the orthonormality of the space.
`CliffordNumbers.jl` aims to provide an implementation of geometric algebras that is fast and easy to use, with a well-documented implementation that can be used as a reference for others who wish to efficiently implement the fundamental operations of geometric algebra. It supports algebras of arbitrary dimension, but the most common anticipated use cases are algebras of low dimensionality, which can take advantage of Julia's compiler infrastructure to produce highly optimized code for common numeric types, including the use of SIMD instructions. Although it is possible to use matrix representations to perform all of the operations of a Clifford algebra, it is rare to require an explicit representation of all linearly independent components, so CliffordNumbers.jl provides small types for handling those cases efficiently. All types representing elements of a Clifford algebra provided by the package subtype `Number` rather than `Array`, allowing them to interoperate seamlessly with existing Julia `Number` types and operations, including the type promotion mechanism. With this architecture, we hope to allow users to generalize algorithms for one-dimensional problems to spaces of higher dimension simply by replacing real numbers with types from CliffordNumbers.jl.

2. Mathematical background

Formally, a *Clifford algebra* $\mathrm{Cl}(V, \mathrm{Q})$ is a unital associative algebra over the vector space V and field K with quadratic form $\mathrm{Q} : V \to K$. Specifically, it is the quotient of the tensor algebra over V, $T(V) = \bigoplus_{x=0}^{\infty} T^x V$, with respect to the two-sided ideal $v \otimes v - \mathrm{Q}(v)\,1$ for all $v \in V$.
For most readers, this definition will likely not be informative without further context. We start with an n-dimensional vector space V, and define the quadratic form as the dot product of vectors with themselves: $\mathrm{Q}(v) = v \cdot v$. The tensor algebra $T(V)$ is the set of all tensor products of elements of V. Each vector in V is a 1-tensor, and the tensor product of k elements is a k-tensor, with the empty product (0-tensors) being scalars K. The space of tensors of grade k over V, $T^k V$, has dimension n^k.
With this, we construct the exterior algebra over V, $\bigwedge(V)$, by taking the quotient of $T(V)$ with respect to the two-sided ideal $v \otimes v$. All tensors of the form $v \otimes v$ are symmetric tensors, and the two-sided ideal eliminates that contribution from the tensor product. This tell us that the product of the exterior algebra, the *wedge product* ($u \wedge v$), is the antisymmetric component of the tensor product $u \otimes v$. It also tells us that the wedge product of linearly dependent of vectors is zero. This limits the grades of $\bigwedge(V)$ to values between 0 and n.
The subspaces of $\bigwedge(V)$ with grade k are denoted $\bigwedge^k(V)$, and their elements are called k-vectors, which have $\binom{n}{k}$ components. A general k-vector is a linear combination of k-*blades*, which are generated by the wedge products of k 1-vectors. The k-blades correspond to oriented regions spanned by the 1-vectors which generated them, and represent primitive geometric objects. For all exterior algebras, scalars, vectors, $n-1$-vectors, and n-vectors are always blades; therefore, all k-vectors are blades in spaces of dimension 3 and below. We can also sum of k-vectors of different grades, producing *multivectors*.
We can now construct the Clifford algebra as a modification of the exterior algebra. The quadratic form Q is simply the dot product of vectors with themselves: $\mathrm{Q}(v) = v \cdot v$. With this, we modify the ideal we used to generate the exterior product from $T(V)$ to $v \otimes v - \mathrm{Q}(v)\,1$. Instead of eliminating the symmetric component of $u \times v$, we replace it a scalar, $u \cdot v$. The *geometric product*, or *Clifford product* of two 1-vectors (denoted without an operator, uv) can now be described succinctly:

$$uv = u \cdot v + u \wedge v \tag{1}$$

While this formula only holds for a very specific case, it may be used to derive the geometric product formula for arbitrary multivectors.
Like k-blades in the exterior algebra, we use the term k-versor to refer to geometric products of 1-vectors. Each versor corresponds to an isometry of space, with 1-versors corresponding to reflections, 2-versors to rotations, and n-versors to inversions, all combined with a scale factor proportional to the magnitude of the versor. In general, k-versors are the sum of blades with grades of the same parity as k, allowing us to classify versors as *even* and *odd*. The preservation of grade parity by the geometric product indicates that Clifford algebras are $\mathbb{Z}_2$-graded algebras.

2.1 Grade operations

Geometric algebras admit a few special operations that manipulate the signs of components of different grades. The *main involution*, or *grade involution* of M, M^*, reverses the signs of all components of odd grade. This is equivalent to a spatial inversion, and changes the orientation of a blade if both the blade and algebra has odd dimension. The *reverse* of M, $M^\dagger$, reverses the signs of all components whose grade *modulo* 2 is odd. This operation is analogous to complex conjugation or the matrix adjoint in that it reverses the order of multiplication:

$$a^\dagger b^\dagger = (ba)^\dagger \tag{2}$$

The *Clifford conjugate* of M is the composition of the main involution and the reverse.
In CliffordNumbers.jl, we overload `Base.reverse` and `Base.adjoint` for the reverse, and we overload `Base.conj`

for the Clifford conjugate. This choice was made to align with the behavior of `Base.adjoint` for matrix multiplication, and to ensure that two possible representations of quaternions in CliffordNumbers.jl (either all elements of $\mathrm{Cl}_{0,2,0}(\mathbb{R})$ or the even elements of $\mathrm{Cl}_{3,0,0}(\mathbb{R})$) behave identically.

2.2 Arithmetic operations

As with ordinary vectors, all elements of a Clifford algebra can be added to each other and scaled by some scalar, so the usual `+`, `-`, `*`, and `/` operators behave as expected. The geometric product also uses the `*` operator, and the wedge product may performed with either the $\wedge$ operator or the `wedge` function. While the quadratic form, geometric product and wedge product are fundamental to geometric algebra, geometric algebras also admit other kinds of operations useful in expressing geometric transformations.

2.2.1 Exponentiation. The square of a multivector is defined as the geometric product of a multivector with itself:

$$a^2 = aa \tag{3}$$

Because the wedge product of any blade with itself is zero, the square of a blade is always a scalar. In algebras with 3 or fewer dimensions, all k-vectors are blades, so they always have scalar squares. But this does not hold in any space of higher dimension: the wedge product of the 2-blade $e_1e_2+e_3e_4$ with itself is not zero, but $2e_1e_2e_3e_4$, since e_1e_2 and e_3e_4 have no factors in commmon. Therefore, the square of this blade must have a non-scalar factor, and it is $2(-1+e_1e_2e_3e_4)$.
From this, all positive integer powers can be defined in terms of the geometric product. CliffordNumbers leverages `Base.literal_pow` to convert the result of exponentiation to a type of minimum size – a detail that may not be inferrable if a naive geometric product is performed.
Additionally, it is also possible to define a natural exponential of a blade b using Taylor series, analogous to complex and quaternion exponentiation. This can be broken down depending on the sign of the square of the blade:

$$e^b = \sum_{n=1}^{\infty} \frac{1}{n!} b^n = \begin{cases} \cos|b| + b\sin|b|, & b^2 < 0 \\ \cosh|b| + b\sinh|b|, & b^2 > 0 \\ 1 + b, & b^2 = 0 \end{cases} \tag{4}$$

Due to the noncommutativity of geometric algebras, the natural exponential of arbitrary multivectors cannot be broken down into a simple sum using the defining property of scalar exponentiation, $e^{a+b} = e^a e^b$.
In the vast majority of cases, the objects to be exponentiated are even multivectors, particularly 2-blades, as these are taken to represent rotations or other types of fundamental transformations, as we will see later.

2.2.2 Inverses and division. Every blade and versor with nonzero square has an inverse:

$$a = \frac{1}{a^2} a^\dagger \tag{5}$$

However, arbitrary elements of a geometric algebra do not necessarily have an inverse, and may instead have zero divisors. CliffordNumbers.jl overloads `inv` with a checked inverse function that throws a `CliffordNumbers.InverseException` if the input does not have a defined inverse.
Because geometric algebras are not commutative, it is also necessary to distinguish between left and right division, and we overload the right division operator `\`, which is already used for matrix operations, to allow these operations to be clearly distinguished.

2.2.3 Duality. The grade n element of an n-dimensional geometric algebra, or *pseudoscalar* (generically written as I), has a special property: multiplying a k-vector by the pseudoscalar produces a $n-k$-vector. This allows us to define the *dual* of a multivector a:

$$\mathrm{dual}(a) = aI^{-1} \tag{6}$$

The dual can be extended to any multivector through the linearity of the geometric product. The reason we use the inverse of the pseudoscalar here is so that we satisfy the following property:

$$a\,\mathrm{dual}(a)^{-1} = I \tag{7}$$

The inverse of I depends on the dimension of the algebra n as well as the quadratic form:

$$I^{-1} = (-1)^{\lfloor \frac{1}{2}n+q \rfloor} I \tag{8}$$

where q is the number of dimensions with negative square. However, the idea of an inverse pseudoscalar breaks down if the Clifford algebra has a degenerate quadratic form.
We can extend our idea of duality if we base it on the wedge product. For a basis blade e, the *right complement* $\overline{e}$ and *left complement* $\underline{e}$ are the basis blades that satisfy the following relations:

$$e \wedge \overline{e} = I \tag{9}$$

$$\underline{e} \wedge e = I \tag{10}$$

As with the pseudoscalar dual, these can be extended by linearity to all multivectors of a geometric algebra.
In CliffordNumbers.jl, the function `pseudoscalar(x)` generates the pseudoscalar of the algebra `x` belongs to. The left contraction and right contraction are provided with the functions `left_contraction` and `right_contraction`, respectively.

2.2.4 Regressive product. The regressive product $a \vee b$ is analogous to the wedge product, but produces objects with complementary grades: if the result of $a \wedge b$ is grade k, the result of $a \vee b$ is grade $n-k$. The regressive product can be defined in terms of the dual in Clifford algebras with nondegenerate quadratic form, but the complements are better suited for defining it in general.

$$\underline{a \vee b} = \underline{a} \wedge \underline{b} \tag{11}$$

$$\overline{a \vee b} = \overline{a} \wedge \overline{b} \tag{12}$$

We will see later that the similarty of these definitions to De Morgan's laws is not a coincidence.

2.2.5 Contractions. The *left contraction* $a \rfloor B$ and *right contraction* $B \lfloor a$ are generalizations of the dot product which blades of differing grades, returning the portion of B remaining when the component spanned by a is removed from it. If a has grade h and B has grade k, the result has grade $k-h$, or if k is less than h, the result is zero. Aside from behaving like a scalar product when one argument is a scalar and a dot product when both arguments have identical grades, the left contraction also satisfies the property

$$(a \wedge b) \rfloor c = a \rfloor (b \rfloor c) \tag{13}$$

which allows us to extend the notion of projection to pairs of blades of any grade [14]. The right contraction is simply the reverse of the left contraction:

$$(a \lfloor b)^\dagger = b^\dagger \rfloor a^\dagger \tag{14}$$

These operations are implemented in CliffordNumbers.jl as operators that can be typed with `\intprod` and `\intprodr`, respectively.

2.3 Transformations

As mentioned previously, k-versors are the geometric product of k 1-blades, and these 1-blades correspond to reflections augmented with a magnitude. This is an algebraic manifestation of the Cartan–Dieudonné theorem, which states that every orthogonal transformation in an n-dimensional Euclidean space is decomposable into no more than n reflections. The 1-blades now come with two interpretations: the usual "arrow" interpretation as objects with magnitude and direction, and a "mirror" interpretation as a plane of reflection, with the arrow corresponding to the direction of reflection.

If we want to reflect the vector v along a mirror plane r, we have to break v into its vector projection and rejection along r, $v_{\parallel r}$ and $v_{\perp r}$. Reflection changes the sign of the projection but preserves the rejection:

$$v = v_{\perp r} + v_{\parallel r} \to v_{\perp r} - v_{\parallel r} \tag{15}$$

The linear algebraic definitions of the projection and rejection are:

$$v_{\parallel r} = r\frac{v \cdot r}{r \cdot r} \tag{16}$$

$$v_{\perp r} = v - r\frac{v \cdot r}{r \cdot r} = v - v_{\parallel r} \tag{17}$$

In geometric algebra, these formulas can be simplified with vector inverses:

$$v_{\parallel r} = r\left(v \cdot r^{-1}\right) \tag{18}$$

$$v_{\perp r} = v - r\left(v \cdot r^{-1}\right) \tag{19}$$

However, the rejection formula can also be expressed in terms of the wedge product. We do this by expanding the geometric product of v with the null product rr^{-1}, which allows us to identify the projection:

$$v = v_{\parallel r} + v_{\perp r} = \left(rr^{-1}\right)v \tag{20}$$

$$= r\left(r^{-1}v\right) \tag{21}$$

$$= r\left(r^{-1} \cdot v + r^{-1} \wedge v\right) \tag{22}$$

$$= r\left(r^{-1} \cdot v\right) + r\left(r^{-1} \wedge v\right) \tag{23}$$

$$v_{\perp r} = r\left(r^{-1} \wedge v\right) \tag{24}$$

We can now construct the reflection of v along r from these components:

$$v_{\perp r} - v_{\parallel r} = r\left(r^{-1} \wedge v\right) - r\left(v \cdot r^{-1}\right) \tag{25}$$

$$= -r\left(v \wedge r^{-1}\right) - r\left(r^{-1} \cdot v\right) \tag{26}$$

$$= -r\left(v \wedge r^{-1} + rr^{-1} \cdot v\right) \tag{27}$$

$$= -rvr^{-1} \tag{28}$$

This double-sided operation is known as a *sandwich product*, the sandwich of v with r.

What if we want to compose two reflections to perform a rotation? We can decompose any rotation R into a pair of reflections, so it is simply a nested sandwich:

$$v_R = -q\left(-rvr^{-1}\right)q^{-1} \tag{29}$$

$$= qrvr^{-1}q^{-1} \tag{30}$$

In this case, the two negative signs from the reflection formula cancel out. Generally, if an orthogonal transformation T consists of k reflections, the sign depends on whether k is even or odd – indicating that a grade involution is being performed. This means that we can construct a more general transformation formula using the grade involution of the transformation:

$$v_T = T^* v T^{-1} \tag{31}$$

$$= Tv\left(T^*\right)^{-1} \tag{32}$$

Although we have derived these formulas assuming that the magnitude of the versor operating on the blade is irrelevant, we can perform transformations that also include scaling if we use the reverse instead of the inverse of T in the sandwich product.

2.4 Families of geometric algebras

For the sake of simplicity, we started by defining our quadratic form as the dot product of a vector with itself. But, in practice, we can make use of non-positive definite or degenerate quadratic forms to produce useful algebras.

2.4.1 Vanilla geometric algebra. A geometric algebra generated with a positive-definite quadratic form is commonly referred to as a *vanilla geometric algebra* (VGA). The term "vector space geometric algebra" was previously used, but is discouraged as all geometric algebras are vector spaces. The most commonly used example is the *algebra of physical space* (APS), $\mathrm{Cl}_{3,0,0}(\mathbb{R})$, whose matrix representation is the Pauli matrices. Vanilla geometric algebras form the basis of the larger modeling algebras we describe here.

2.4.2 Spacetime and Lorentzian geometric algebra. The algebra of physical space can be extended to the *spacetime algebra* (STA), $\mathrm{Cl}_{1,3,0}(\mathbb{R})$ or $\mathrm{Cl}_{3,1,0}(\mathbb{R})$ depending on the choice of metric signature. Although these algebras are not isomorphic, their even subalgebras are both $\mathrm{Cl}_{3,0,0}(\mathbb{R})$, the algebra of physical space [24]. in general, the even subalgebras of both $\mathrm{Cl}_{1,n,0}(\mathbb{R})$ or $\mathrm{Cl}_{n,1,0}(\mathbb{R})$ are $\mathrm{Cl}_{n,0,0}(\mathbb{R})$.

Odd blades of STA can be projected APS with the use of a *spacetime split*: by multiplying any 1-blade of STA with the timelike vector γ_0, the result is projected into the even subalgebra, with timelike components becoming scalars and spacelike components becoming 1-blades of APS. This operation naturally maps relativistic vector quantities (such as the electromagnetic four-potential) onto classical pairs of scalar and vector quantities (such as voltage and the magnetic potential). Along with ordinary VGA rotors, rotors constructed from a bivector with a timelike component have a natural interpretation as Lorentz boosts. In this work and in CliffordNumbers.jl, we use the general term *Lorentzian geometric algebra* (LGA) for algebras that combine a single timelike dimension with multiple spacelike dimensions.

2.4.3 Projective and plane-based geometric algebra. If we extend an n-dimensional vanilla geometric algebra with a single degenerate dimension spanned by a basis vector e_0, we obtain a projective geometric algebra (PGA), $\mathrm{Cl}_{n,0,1}(\mathbb{R})$. The original formulation of this model uses 1-blades to describe points, but the alternative mirror space convention is preferred in more recent literature due to the fundamental role that reflections play in describing transformations, and this specific approach is commonly called *plane-based* geometric algebra [13]. PGA is a homogeneous model, meaning that scalar multiples of blades represent equivalent geometry, identically to how multiplying both sides of the equation of a plane $ax + by + cz = d$ by the same constant does not change the plane represented by the equation. The wedge product of blades is commonly called the *meet*, as it represents the intersection of planes, or equivalently, an underdetermined linear system. Similarly, the regressive product is the *join*, as it unites the sub-

spaces spanned by two or more blades. In n-dimensional PGA, the wedge product of $n-1$ 1-blades is a line. The description of lines in 3D space as bivectors is exactly identical to Plücker coordinates [27].

The versors of PGA correspond to elements of the Euclidean group $\mathrm{E}(n)$: the exponentiation of bivectors can produce the usual VGA rotors, but bivectors with the degenerate e_0 component produce translations. We refer to motors with a translation component *motors*, and this term includes pure translations as well as screw motions, their composition with rotations. It is also possible to construct projective geometric algebras with modeling dimensions that do not square to zero, and the sign of the square of the modeling dimension corresponds to the curvature of the space [20]. This allows us to reinterpret n-dimensional VGA as a homogeneous model of geometry on an n-sphere.

2.4.4 Conformal geometric algebra. Conformal geometric algebra (CGA) augments a vanilla geometric algebra with two extra dimensions, one positive-squaring and one negative-squaring, corresponding to the Clifford algebra $\mathrm{Cl}_{n+1,1,0}(\mathbb{R})$ [11]. The two extra modeling dimensions are spanned by the basis 1-blades, which we denote here as e_- and e_+, squaring to -1 and $+1$, respectively. However, it is usually more convenient to use a basis of two null (zero-squaring) 1-blades, n_0 and n_∞, corresponding to the point at the origin and the point at infinity:

$$n_0 = \frac{1}{2}(e_- - e_+) \tag{33}$$

$$n_\infty = e_- + e_+ \tag{34}$$

The 1-blades of CGA are $(n-1)$-spheres, and like in PGA, the wedge product of k 1-blades corresponds to the intersections between k spheres (or a k-pencil of spheres), which are also spheres of lower dimension [5]. However, these spheres may have radius 0 (points), infinite radius (flat planes), or even an imaginary radius, supplying information about the orientation of the object or how two objects intersect. A sphere of radius r and with coordinates (x, y, z) is given by a blade of the form

$$b = w\left(xe_1 + ye_2 + ze_3 + wn_0 + \frac{x^2+y^2+z^2-r^2}{2}n_\infty\right) \tag{35}$$

with w being a weight factor irrelevant to the geometry, as CGA is also a homogeneous model. The squared radius of a pencil of spheres, p, is given by the equation

$$\mathrm{rad}^2(p) = pp^\dagger\left((n_\infty \rfloor p)(n_\infty \rfloor p)^\dagger\right)^{-1} \tag{36}$$

All compass-and-straightedge constructions can be performed in 2D CGA, and these can be generalized to any number of dimensions. CGA can also represent all of the elements of PGA in a succinct way: for example, a hyperplane can be thought of as an n-sphere of infinite radius, and a straight line can be thought of as a defined by three points, one of which is the special point at infinity, n_∞. When linking the two formalisms, the unique spherical objects of CGA are referred to as "rounds", and the elements that are present in PGA are "flats". CGA can also represent the rotors and motors of PGA, but it also allows for arbitrary conformal transformations to be constructed.

3. Software Architecture

The implementation of geometric algebras in this package generally follows that of Geometric Algebra for Computer Science by Dorst, Fontijne, and Mann [15], as it was started as a simple exercise in the author's spare time before becoming a full-fledged package.

CliffordNumbers.jl provides the abstract type `AbstractCliffordNumber{Q,T}`, a subtype of `Number` which an element of the Clifford algebra `Q` and the real or complex type `T`. The concrete types provided include `CliffordNumber{Q,T,L}`, which explicitly represents every element of the algebra, `EvenCliffordNumber{Q,T,L}` and `OddCliffordNumber{Q,T,L}` which only explicitly represent the even or odd grade elements, and `KVector{K,Q,T,L}` which only explicitly represents elements of grade `K`. The type parameter `L` stores the length of the `Tuple` that backs each of these types, and can be omitted in most contexts.

Unlike subtypes of `Number` provided by Julia, construction and conversion of Clifford numbers have different semantics. The construction of a type that explicitly represents fewer grades from one that does explicitly represent those grades (for instance, `EvenCliffordNumber` from `CliffordNumber`) will perform grade projection, implicitly setting all elements not represented by the result type to 0. On the other hand, conversion will throw a `InexactError` if any grade not representable by the result type is not zero.

3.1 Algebras

In a purely mathematical sense, all Clifford algebras can be uniquely identified by their metric signature. However, we often use algebras with identical metric signature to represent distinct geometries: $\mathrm{Cl}_{3,1,0}(\mathbb{R})$ could represent either the spacetime algebra with mostly positive metric signature (East Coast convention), or the 2D conformal geometric algebra. Although the mathematical operations defined in these algebras is identical, it may be useful to handle these cases differently: for instance, when pretty printing their elements or plotting blades of these algebras.

Code for handling the specification of an algebra is found in the `Metrics` submodule. Aside from the `AbstractSignature` and generic `Signature` types, this submodule provides types for most commonly used algebras. `AbstractSignature` is a subtype of `AbstractVector{Int8}` which takes on values of either $+1$, -1, or 0, corresponding to the metric signature. In essence, `AbstractSignature` objects are the diagonals of metric tensors. The indices of an `AbstractSignature` are a `UnitRange`, representing the indices of basis blades for an algebra defintion. This allows us to produce nicer printed representations of algebras with e_0 components, like STA or PGA.

3.2 Indexing

Because `AbstractCliffordNumber` subtypes are not arrays, indexing one with integer indices behaves as it does with other subtypes of `Number`. Indexing with any number of ones will return itself, and indexing with any other index returns a `BoundsError`. To avoid conflict with the established behavior of number indexing, we provide the `BitIndex{Q}` type which represents all basis blades of the algebra `Q`.

`BitIndex{Q}` is backed by an `Int` whose bits represent the presence or absence of particular basis 1-blades in the indexed component. The sign bit of the `Int` represents the sign of the permutation of the indices. When constructing a `BitIndex{Q}` from the integer indices of the basis blades, the parity is used to determine the sign, and it may be flipped with negation.

The coefficients of a `CliffordNumber` are stored in the natural binary order imparted by `BitIndex{Q}`. For `EvenCliffordNumber`

Table 1. : Types and aliases for commonly used geometric algebras

Exported name	Algebras
`VGA`	Vanilla geometric algebras ($\mathrm{Cl}_{n,0,0}(\mathbb{R})$)
`PGA`	Projective/plane-based geometric algebras ($\mathrm{Cl}_{n,0,1}(\mathbb{R})$)
`CGA`	Conformal geometric algebras ($\mathrm{Cl}_{n+1,1,0}(\mathbb{R})$)
`LGA`	Supertype for all Lorentzian geometric algebras
`LGAWest`	Lorentzian geometric algebras with mostly negative signature ($\mathrm{Cl}_{1,n,0}(\mathbb{R})$)
`LGAEast`	Lorentzian geometric algebras with mostly positive signature ($\mathrm{Cl}_{n,1,0}(\mathbb{R})$)
`Exterior`	Exterior algebras ($\mathrm{Cl}_{0,0,n}(\mathbb{R})$)
`STA`, `STAWest`	Spacetime algebra with mostly negative signature (alias for `LGAWest(3)`)
`STAP`, `STAPWest`	Spacetime algebra with mostly negative signature (alias for `LGAWest(3)`)
`STAEast`	Spacetime algebra with mostly positive signature (alias for `LGAEast(3)`)

and `OddCliffordNumber`, the natural binary ordering can still be used efficiently: the integer indices of either of these types can be mapped to the indices of a `CliffordNumber` by adding an extra bit to the index and setting it to zero or one to make the Hamming weight of the integer even or odd. The case of `KVector` is the most complicated, but is implemented in a generated function to unroll any loops with with size determined by the type.
Additionally, we provide the `BitIndices{Q,C}` type, a statically sized vector all of the valid `BitIndexQ` object for a Clifford number of type `C`. This type is used in the implementation of various algorithms that iterate over the scalar coefficients of multivectors, incluiding the products of geometric algebra. On top of this, we provide another type, `TransformedBitIndices`, which lazily computes a transformation to a `BitIndices` object.

3.3 Implementation of products

Internally, CliffordNumbers.jl provides the function `CliffordNumbers.mul`, which takes two multivectors as an argument along with an instance of the singleton type `CliffordNumbers.GradeFilter`. The `GradeFilter` object represents the rules for incorporating the product of basis blade coefficients into the final result. For instance, the wedge product omits any components which are not linearly independent, so a specialization on `GradeFilter{:∧ }` ensures that the results of those products are set to zero. The geometric product is special in that no rejection of coefficient products is performed.
Calculating the geometric product starts with the rules for multiplying orthonormal basis 1-blades:

$$e_i e_j = \begin{cases} e_i \wedge e_j, = -e_j \wedge e_i, & i \neq j \\ e_i \cdot e_j, & i = j \end{cases} \tag{37}$$

It is trivial to reproduce this behavior with the `BitIndex` objects we introduced earlier by simply performing `xor` on their bits. The only remaining task is obtaining the correct sign, as that depends on the permutation of input blades. To calculate the geometric product, we iterate through the `BitIndices` of one multivector, using each `BitIndex` to shuffle the coefficients and flip the signs of the other multivector. The shuffle result is then scaled by the coefficient of the multivector associated with that `BitIndex` and accumulated in the result by summation.

Because modern CPUs with SIMD extensions implement shuffle operations and vectorized fused multiply-add (fma), the geometric product can be calculated very efficiently. All blades and versors of APS can fit into a single 256-bit register (as in Intel AVX2) if the scalar coefficients are double-precision floats, and the same is true of STA, 3D PGA, and 2D CGA if the scalar coefficients are single-precision floats. However, we cannot write this code using `for` loops, vectorization, or `map`, and take advantage of SIMD instructions, even with heavy use of `@inline`.
To accomplish this, we need to manually unroll the operations using generated functions, allowing Julia's compiler to emit optimized SIMD code for any pair of arguments. This unrolling is sensitive to the lengths of the input multivectors, and will preferentially loop over the indices of the smaller multivector so that there are fewer total iterations and the full SIMD register width is used when possible. The use of generated functions also allows us to pre-compute all constants used in the multiplication (such as shuffle indices) at compile time.

3.4 Interoperation with other `Number` types

Julia uses type promotion heavily with nearly all of its numerical operations, allowing for generic operator defintions that work with a wide variety of inputs, even if the types are mismatched. CliffordNumbers.jl defines promotion rules for operations that combine `AbstractCliffordNumber` subtypes with Julia's base `Number` types, `Real` and `Complex`. One important decision is that almost all operations that return scalar values with `AbstractCliffordNumber{Q}` inputs will return a `KVector{0,Q}`, so that information about the algebra `Q` is propagated to the result.
Because `AbstractCliffordNumber{Q,T}` is not an `AbstractArray` or otherwise iterable object, calling `eltype(x::AbstractCliffordNumber)` returns `typeof(x)` as with ordinary numbers, and `length(x::AbstractCliffordNumber)` is always 1. For this reason, we provide the functions `scalar_type` and `nblades` to obtain this information without interfering with the expected behavior of `eltype` and `length`.

4. Usage examples

Figures accompanying the examples were generated with Makie.jl [7]. Makie.jl does not have built-in support for natively plotting the types present in CliffordNumbers.jl, so we wrote our own functions for plotting various elements of geometric algebras as different types of objects. While CliffordNumbers.jl does not intend to impose a particular geometric interpretation on the types it provides, integrating plotting support though a package extension is a future goal for the package.

4.1 Making multivectors plottable

Makie.jl uses the GeometryBasics.jl package internally, which provides definitions for how different types of objects should be plotted. The most important types for plotting will be `Point{N,T}` and `Hypersphere{N,T}`, which are recognized by Makie.jl for various common plot types, such as scatter plots.
In VGA, converting 1-blades to `Point` objects is trivial. Converting an $(n-1)$-blade of VGA to `Point` is also relatively easy, but we have to factor out the null component. and center.

Code 1: Converting VGA 1-blades and and PGA $n-1$-blades to GeometryBasics points.

```
using CliffordNumbers
using GeometryBasics, StaticArrays

function Point(x::KVector{1,Q}) where Q
    Q isa VGA || error("input must be a VGA element.")
    return SVector(Tuple(x))
end

function Point(x::KVector{K,Q}) where {K,Q}
    D = dimension(Q)
    (Q isa PGA && K = D - 1) || error("input needs to have grade $(D - 1) for PGA elements.")
    return SVector(Base.tail(Tuple(x))) / x[BitIndex(x, 0)]
end
```

Converting a CGA 1-blade to a hypersphere requires us to recover the radius and center.

Code 2: Converting 1-blades of CGA to GeometryBasics.jl hyperspheres.

```
using CliffordNumbers
using GeometryBasics, StaticArrays

function ninf(x::AbstractCliffordNumber{Q,T}) where {Q,T}
    Q isa CGA || error("input must be a CGA element.")
    return KVector{1,Q}(one(T), one(T), ntuple(Returns(zero(T)), Val(dimension(Q)))...)
end

function radius2(x::KVector)
    w = left_contraction(ninf(x), x)
    return (x * x') * inv(w * w')
end

function center_vector(x::KVector{1,Q}) where Q
    w = left_contraction(ninf(x), x)
    return SVector{dimension(Q)}((x / w)[i] for i in BitIndices(x)[3:end])
end

function GeometryBasics.Hypersphere(x::KVector{1,Q,T}) where Q
    N = dimension(Q)
    return Hypersphere{dimension(Q),T}(center_vector(x), sqrt(radius2(x)))
end
```

The most difficult challenge is plotting the 1-blades of 2D CGA as hyperplanes. Makie.jl does not have a built-in function to directly plot a line of the form $ax + by = c$, but it does have the `ablines` function for plotting a line in slope-intercept form, $y = mx + b$. We can transform our coefficients to produce the correct form for `ablines`, but vertical lines will not plot at all.

Code 3: Plotting 1-blades of PGA to GeometryBasics.jl hyperspheres.

```
using CliffordNumbers
using Makie

function Makie.convert_arguments(
    ::Type{<:ABLines},
    xs::AbstractVector{<:KVector{1,PGA(2)}}
)
    slopes = [-x[BitIndex(x, 1)] / x[BitIndex(x, 2)] for x in xs]
    intercepts = [x[BitIndex(x, 0)] / x[BitIndex(x, 2)] for x in xs]
    return (intercepts, slopes)
end
```

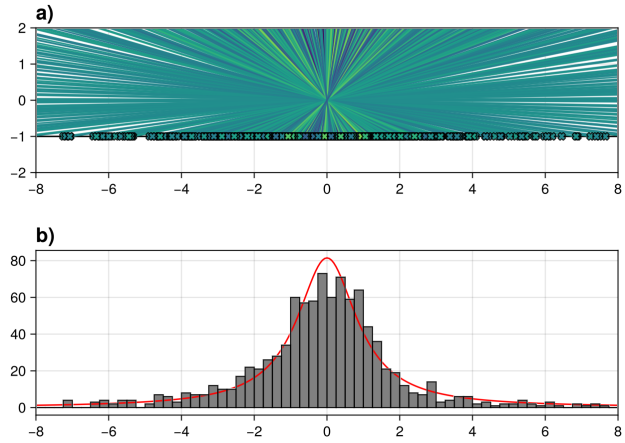

Fig. 1: a) Points generated by the intersection of random rays with uniformly distributed angles with a line. b) Histogram of the x-value of the points, accompanied by their histogram and expected shape of their distribution.

4.2 The Cauchy distribution

The Cauchy probability distribution is given by:

$$\text{Cauchy}(x, x_0, \gamma) = \frac{1}{\pi\gamma\left[1 + \left(\frac{x - x_0}{\gamma}\right)\right]} \tag{38}$$

where x_0 is the median and γ is the median absolute deviation. While the Cauchy distribution is pathological in that it has no mean or variance, it does arise naturally as the ratio distribution of normally distributed random variables with zero mean.
One of the most common constructions of the Cauchy distribution is from the intersection of uniformly distributed random rays with a plane. This process can be illustrated easily with 2D PGA. We generate random rays through the origin, then find their meet by taking their wedge product with the line at $y = -1$:

Code 4: Ray-line intersections.

```
using CliffordNumbers

n = 1024    # change this to taste
rays = [KVector{1,PGA(2)}(0, randn(), randn()) for _ in 1:n]
line = KVector{1,PGA(2)}(-1, 1, 0)
intersections = wedge.(line, rays)
```

The multidimensional Cauchy distribution also arises naturally in n-dimensional PGA by generating $n-1$-blades with coefficients drawn from a standard normal distribution. We recover the coordinates of a point by dividing all other blade coefficients by the

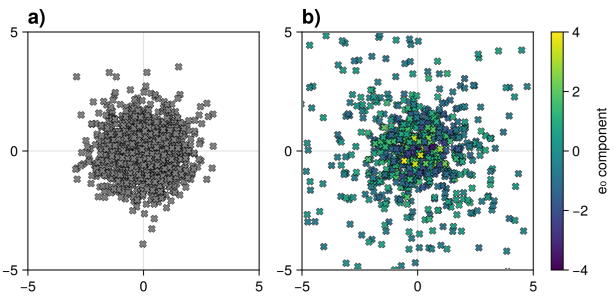

Fig. 2: a) 1024 points drawn from a multivariate standard normal distribution ($\mu = [0, 0], \sigma_{ij} = \delta_{ij}$). b) 1024 PGA 2-blades with all coefficients drawn from a standard normal distribution, producing a multivariate Cauchy distribution with fatter tails than the normal distribution.

e_0 coefficient, which generates the Cauchy distribution as a ratio distribution.

Code 5: The multidimensional Cauchy distribution.

```
using CliffordNumbers

n = 1024    # change this to taste
points = [KVector{2,PGA(2)}(randn(), randn(),
    randn()) for _ in 1:n]
```

This illustrates that the Cauchy distribution is intrinsically linked to the idea of projection, whether we do so by explicitly tracing the intersection of rays onto a plane, or implicitly with the use of a projective geometric algebra.

4.3 Intersections of circles and spheres

Finding intersections between spheres of various dimensions is a critical step in the construction of simplicial complexes and filtrations used in topological data analysis (TDA). To construct a filtration, we take a point cloud and select a range of radii to filter the data with. We can do this easily with CGA tooling: along with a function for generating CGA spheres, we also include a function for measuring their squared radii.

Code 6: Constructing and measuring CGA spheres.

```
using CliffordNumbers
using StaticArrays

function ninf(x::AbstractCliffordNumber{Q,T})
    where {Q,T}
    Q isa CGA || error("input must be a CGA
        element.")
    return KVector{1,Q}(one(T), one(T), ntuple(
        Returns(zero(T)), Val(dimension(Q)))...)
end

function CGA_sphere(point::StaticVector{D}, radius
    ) where D
    # add n0 to the coordinates
    x = KVector{1,CGA(D)}(1//2, -1//2, point...)
    # construct n\infty
    return x + 1//2 * (sum(point.^2) - radius^2) *
         ninf(x)
end

function radius2(x::KVector)
    w = left_contraction(ninf(x), x)
    return (x * x') * inv(w * w')
end
```

In most cases, we use a common filtration radius r across all points when constructing a complex, which allows us to rephrase the question as determining whether the minimum bounding sphere of a group of k points is less than or equal to r.
Welzl's algorithm can be used to determine the minimum bounding sphere of a set of k points in $\mathrm{O}(k)$ time on average [30]. As part of this algorithm, we may need to construct spheres that pass through up to $n + 1$ points in an n-dimensional space. This can be accomplished by taking the wedge product of up to $n + 1$ points, represented by spheres of zero radius. The initial result of this operation is not necessarily an $(n - 1)$-sphere. But given a CGA round r, we can always construct a sphere with the same center and radius of r (called the *container* of r)[1] using the formula

$$\mathrm{container}(r) = \left(\left(rI^{-1}\right) \wedge n_\infty\right) \vee r \tag{39}$$

With CliffordNumbers.jl, we can define a function to perform this construction as part of Welzl's algorithm.

Code 7: Constructing a sphere from any round.

```
using CliffordNumbers

function CGA_container(r::KVector{K,Q}) where {K,Q
    }
    Q isa CGA || error("input blade must be an
        element of a CGA.")
    # construct n\infty
    ninf = KVector{1,CGA(D)}(1, 1, zero(point)...)
    return wedge(wedge(r * pseudoscalar(r), ninf),
        r)
end
```

This facilitates constructing a sphere that passes through less than $n + 1$ points, regardless of the number of spatial dimensions.

5. Limitations

Geometric algebra is a promising alternative approach to many problems in numerical computing, but is not a universal solution to every problem. In theory, it is possible to use the Clifford algebra $\mathrm{Cl}_{n,n,0}(\mathbb{R})$ to represent any n-dimensional matrix [12], and therefore perform any operation in linear algebra. But in practice, the number of operations needed to implement such an approach far exceeds that of ordinary matrix multiplication. We still recommend the use of linear algebra libraries when dealing with objects of very high dimensionality.

5.1 Large algebras

Operations on the `Tuple` type are subject to inlining limits imposed by Julia. In practice, this means that the best performance is limited to algebras generated by a six-dimensional vector space. In practice, this will be sufficient for almost all work with commonly used algebras, including 2D and 3D VGA, PGA and CGA, as well as the spacetime algebra. Future work on the Julia compiler, particularly on the implementation of `NTuple`, may improve on these bounds.

[1] If CGA 1-blades are taken to be points, this construction is called the *center*.

5.2 Operator precedence and associativity

In some published literature [10][22] and some software packages in different languages (such as galgebra)[3], the geometric product is generally given lower precedence than the wedge product, which is lower than either contractions or dot products. However, these conventions do not appear to be standardized, and precedence may not even be defined for some operations, like the regressive product. In Julia, the list of allowed operators and their precedences are hard-coded into the language. The function `Base.operator_precedence` can be used to determine the precedence of operators, taking a `Symbol` as input and returning an `Int` corresponding to the precedence. Higher precedence operators take higher values, and those with equal precedence are evaluated from left to right. The majority of product operators have precedence 12, with $\vee$ used for the regressive product taking precedence 11, so operators are evaluated sequentually other than the regressive product. For this reason, we strongly recommend explicitly using parentheses when ordering multiplicative operators in code.

5.3 Numerical stability

Not all operations in this package are necessarily numerically stable. In particular, this is true of the wedge product: repeated wedge products can accumulate significant error [9]. We plan to mitigate this in the case of wedging n 1-blades together by using the techinques from linear algebra suggested by de Keninck et. al., but we also seek to improve numerical stability when wedging fewer than n blades together.

However, this should not be taken to mean that geometric algebra is necessarily an inferior choice where numerical stability is critical. The choice of approach should factor in the different ways error can manifest in a particular application. In cases where the preservation of orthonormality is critical, transforms built with versor composition can never introduce shearing, unlike matrix-based methods.

6. Future work

The highest priorities in future releases of CliffordNumbers.jl is ensuring that package methods are numerically stable and providing performant implementations of commonly used sequences of operations, such as the sandwich product. While the topic of numerical stability for operations in geometric algebra has not been thoroughly explored, we may use existing techniques from linear algebra or develop analogous modifications to naive algorithms. Of interest is the ability to express the Gram-Schmidt process in terms of wedge products [10], since the naively implemented Gram-Schmidt process is numerically unstable, but admits modifications and alternative algorithms that are stable. We plan to extend CliffordNumbers.jl to interoperate with more of the Julia ecosystem, particularly Julia's standard libraries. Package extensions already exist for the LinearAlgebra standard library, StaticArrays.jl, and Quaternions.jl to encourage interoperation with existing code. One of our primary goals is to provide support for using CliffordNumbers.jl with the extensive Julia ecosystem for automatic differentiation (AD). The implementation of forward mode automatic differentiation may be facilitated by the fact that dual numbers are the Clifford algebra $\mathrm{Cl}_{0,0,1}(\mathbb{R})$, and dualization of the components of the algebra can be accomplished simply by adding a degenerate dimension. This will facilitate the use of multivector types from CliffordNumbers.jl as drop-in replacements for ordinary scalar weights. Additionally, work has begun on integrating `AbstractCliffordNumber` with Julia's random number generation functionality, including methods for `randn` and `rand` for 1-blades of VGA, PGA, and CGA. This work will be extended to the Distributions.jl package [2] to support other useful distributions of blades or rotors over Clifford algebras. In the far future, we hope to work on providing an implementation of the Clifford-Fourier transform [16]. This transform generalizes the Fourier transform to arbitrary dimensions without the need for complex numbers. Along with a single-sided transform that generalizes the bivector properties of the complex numbers, a double-sided transform can also be constructed from the sandwich product [25].

7. Acknowledgements

CliffordNumbers.jl is not the only geometric algebra package for Julia. The authors would like to acknowledge existing geometric algebra packages for Julia, including GeometricAlgebra.jl by Joseph Wilson (Jollywatt) [31], CliffordAlgebras.jl by Andreas Tell (ATell-SoundTheory) [29], Multivectors.jl by Michael Alexander Ewert (digitaldomain) [17], Grassmann.jl by Michael Reed (chakravala) [28], and SymbolicGA.jl by Cédric Belmant (serenity4) [1].

B. S. Flores would like to acknowledge the members of the geometric algebra community at bivector.net. Specifically, this includes Adam Jarhaus (Kruglord), Steven de Keninck (enki), Hamish Todd, Moab Croft (Slackline), and David Cohoe (sudgylacmoe), who produces educational videos about geometric algebra, including the video that introduced him to the field, *A Swift Introduction to Geometric Algebra* [8]. B. S. Flores would also like to acknowledge Chris Doran, Mason Protter, and Cédric Belmant (serenity4) for discussing the implementation of Clifford algebras in Julia, as well as Prof. Mateusz Baran for discussion of the related StaticArrays.jl package. Finally, B. S. Flores would like to acknowledge Dr. Jessica Geisenhoff for restoring his love for math after numerous challenging undergraduate classes, and Dr. Michael Davies for bringing him a deeper appreciation for computing on all levels, from hardware to the human.

8. References

[1] Cédric Belmant. serenity4/SymbolicGA.jl, January 2026. original-date: 2022-11-12T12:34:23Z.

[2] Mathieu Besançon, Theodore Papamarkou, David Anthoff, Alex Arslan, Simon Byrne, Dahua Lin, and John Pearson. Distributions.jl: Definition and modeling of probability distributions in the juliastats ecosystem. *Journal of Statistical Software*, 98(16):1–30, 2021. doi:10.18637/jss.v098.i16.

[3] Alan Bromborsky, Utensil Song, Eric Wieser, Hugo Hadfield, and The Pygae Team. pygae/galgebra, June 2020. doi:10.5281/zenodo.3857096.

[4] James M. Chappell, Azhar Iqbal, John G. Hartnett, and Derek Abbott. The vector algebra war: A historical perspective. *IEEE Access*, 4:1997–2004, 2016. doi:10.1109/ACCESS.2016.2538262.

[5] Clément Chomicki, Stéphane Breuils, Venceslas Biri, and Vincent Nozick. Pencils and Set Operators In 3D CGA. May 2024.

[6] William Kingdon Clifford. Applications of grassmann's extensive algebra. *American Journal of Mathematics*, 1(4):350–358, 1878.

[7] Simon Danisch and Julius Krumbiegel. Makie.jl: Flexible high-performance data visualization for Julia. *Journal of Open Source Software*, 6(65):3349, 2021. doi:10.21105/joss.03349.

[8] David Cohoe (sudgylacmoe). A Swift Introduction to Geometric Algebra, August 2020.

[9] Steven De Keninck and Leo Dorst. Hyperwedge. In *Advances in Computer Graphics: 37th Computer Graphics International Conference, CGI 2020, Geneva, Switzerland, October 20–23, 2020, Proceedings*, page 549–554, Berlin, Heidelberg, 2020. Springer-Verlag. doi:10.1007/978-3-030-61864-3_47.

[10] C. Doran and A. Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, 2007.

[11] Chris Doran, Anthony Lasenby, and Joan Lasenby. *Conformal Geometry, Euclidean Space and Geometric Algebra*, pages 41–58. Springer US, Boston, MA, 2002. doi:10.1007/978-1-4615-0813-7_4.

[12] Leo Dorst. 3d oriented projective geometry through versors of

$$\mathbb{R}^{3,3}$$

. *Advances in Applied Clifford Algebras*, 26(4):1137–1172, December 2016. doi:10.1007/s00006-015-0625-y.

[13] Leo Dorst and Steven De Keninck. A Guided Tour to the Plane-Based Geometric Algebra PGA. March 2022.

[14] Leo Dorst, Chris Doran, and Joan Lasenby. *The Inner Products of Geometric Algebra*, pages 35–46. Birkhäuser Boston, Boston, MA, 2002. doi:10.1007/978-1-4612-0089-5_2.

[15] Leo Dorst, Daniel Fontijne, and Stephen Mann. *Geometric Algebra for Computer Science: An Object-Oriented Approach to Geometry*. The Morgan Kaufmann Series in Computer Graphics. Morgan Kaufmann, 2010.

[16] J. Ebling and G. Scheuermann. Clifford fourier transform on vector fields. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):469–479, 2005. doi:10.1109/TVCG.2005.54.

[17] Michael Ewert. digitaldomain/Multivectors.jl, June 2025. original-date: 2020-02-25T22:32:49Z.

[18] J. Willard Gibbs. Multiple algebra. *Science*, ns-8(186):180–181, 1886. doi:10.1126/science.ns-8.186.180. https://www.science.org/doi/pdf/10.1126/science.ns-8.186.180.

[19] H. Grassmann. *Die lineale Ausdehnungslehre ein neuer Zweig der Mathematik: dargestellt und durch Anwendungen auf die übrigen Zweige der Mathematik, wie auch auf die Statik, Mechanik, die Lehre vom Magnetismus und die Krystallonomie erläutert*. Wissenschaft der extensiven Grösse. O. Wigand, 1844.

[20] Charles G. Gunn. Doing Euclidean Plane Geometry Using Projective Geometric Algebra. *Advances in Applied Clifford Algebras*, 27(2):1203–1232, June 2017. doi:10.1007/s00006-016-0731-5.

[21] William Rowan Hamilton. Lxxviii. on quaternions; or on a new system of imaginaries in algebra. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 25(169):489–495, 1844. doi:10.1080/14786444408645047. https://doi.org/10.1080/14786444408645047.

[22] D. Hestenes and G. Sobczyk. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. Fundamental Theories of Physics. Springer Netherlands, 2012.

[23] David Hestenes. Real spinor fields. *Journal of Mathematical Physics*, 8(4):798–808, 04 1967. doi:10.1063/1.1705279. https://pubs.aip.org/aip/jmp/article-pdf/8/4/798/19138621/798_1_online.pdf.

[24] David Hestenes. Spacetime physics with geometric algebra. *American Journal of Physics*, 71(7):691–714, 07 2003. doi:10.1119/1.1571836. https://pubs.aip.org/aapt/ajp/article-pdf/71/7/691/8504660/691_1_online.pdf.

[25] Eckhard Hitzer. General Steerable Two-sided Clifford Fourier Transform, Convolution and Mustard Convolution. *Advances in Applied Clifford Algebras*, 27(3):2215–2234, September 2017. doi:10.1007/s00006-016-0687-5.

[26] P. Lounesto. *Clifford Algebras and Spinors*. Cambridge Handbooks for Language Teachers. Cambridge University Press, 2001.

[27] D. B. Nguyen and Garret Sobczyk. Notes on Plücker's relations in geometric algebra. *Advances in Mathematics*, 363:106959, March 2020. doi:10.1016/j.aim.2019.106959.

[28] Michael Reed. Differential geometric algebra with Leibniz and Grassmann. *JuliaCon*, 1(1):1–6, January 2019. doi:10.5281/zenodo.101519786.

[29] Andreas Tell. CliffordAlgebras.jl: Fast Clifford/Geometric Algebra in Julia, 2025. Publication Title: Zenodo original-date: 2021-11-22T14:53:11Z.

[30] Emo Welzl. Smallest enclosing disks (balls and ellipsoids). In Hermann Maurer, editor, *New Results and New Trends in Computer Science*, pages 359–370, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg.

[31] Joseph Wilson. Jollywatt/GeometricAlgebra.jl, February 2026. original-date: 2020-09-18T00:14:32Z.